

Agent Run Modes

Hermes Agent Run Modes

1. Overview

Hermes Agent is a full-featured multimodal AI Agent system that supports multiple run modes for different scenarios (local CLI interaction, remote Gateway service, Honcho persistent memory integration, etc.). Unlike the common "Agent mode vs Plan mode" dichotomy, Hermes distributes the concept of **modes** across multiple independent dimensions, including: API call mode, memory write mode, memory recall mode, write frequency mode, CLI input mode, session strategy mode, reasoning level, and more. These dimensions can be freely combined, forming a multi-dimensional configuration space.

Core File List:

File	Purpose
run_agent.py	AIAgent main class, core execution logic for all modes
honcho_integration/client.py	Honcho client configuration, memory/recall mode definitions
honcho_integration/cli.py	Honcho CLI command handling
honcho_integration/session.py	Honcho Session manager
hermes_cli/main.py	CLI main entry, honcho subcommand registration
gateway/run.py	Gateway service, /plan command entry
agent/skill_commands.py	/plan and other slash command construction logic
toolsets.py	Toolset system definition
hermes_state.py	SQLite session persistence storage
cli.py	Legacy CLI entry, display/tool_progress modes
gateway/config.py	Gateway platform configuration

2. API Call Mode (api_mode)

2.1 Mode Definition

Location: run_agent.py:587–609 , api_mode attribute in AIAgent.__init__

API call mode determines how Hermes communicates with the underlying LLM provider. Three mutually exclusive modes:

Mode	Value	Description
OpenAI Compatible Chat Completions	chat_completions	Standard OpenAI /v1/chat/completions API, default value
OpenAI Codex Responses	codex_responses	OpenAI Responses API (for GPT series models, supports richer tool calling)
Anthropic Native Messages	anthropic_messages	Anthropic /v1/messages API (for native Claude calls)

2.2 Auto-Detection Logic

```
# run_agent.py AIAgent.__init__ source logic
if api_mode in {"chat_completions", "codex_responses", "anthropic_messages"}:
```

```

        self.api_mode = api_mode          # Explicitly passed
    elif self.provider == "openai-codex":
        self.api_mode = "codex_responses" # OpenAI Codex provider
    elif "chatgpt.com/backend-api/codex" in self._base_url_lower:
        self.api_mode = "codex_responses" # Codex URL
    elif self.provider == "anthropic" or "api.anthropic.com" in self._base_url_lower:
        self.api_mode = "anthropic_messages" # Anthropic direct call
    elif self._base_url_lower.rstrip("/").endswith("/anthropic"):
        self.api_mode = "anthropic_messages" # Third-party compatible endpoint (MiniMax, DashScope,
        etc.)
    else:
        self.api_mode = "chat_completions" # Default: OpenAI compatible

# Direct OpenAI URL → upgrade to Responses API (GPT-5.x tool calling rejects chat/completions)
if self.api_mode == "chat_completions" and self._is_direct_openai_url():
    self.api_mode = "codex_responses"

```

2.3 Impact on Core Behaviors

2.3.1 Prompt Caching Enable Conditions

```

# run_agent.py:667-669
is_openrouter = self._is_openrouter_url()
is_claude = "claude" in self.model.lower()
is_native_anthropic = self.api_mode == "anthropic_messages"
self._use_prompt_caching = (is_openrouter and is_claude) or is_native_anthropic

```

- `anthropic_messages` mode: always enabled
- OpenRouter + Claude model: enabled (via OpenRouter's cache primitive)
- `chat_completions` / `codex_responses` : not enabled

2.3.2 API Request Construction Differences

```

# run_agent.py:4617-4660
if self.api_mode == "anthropic_messages":
    # Use Anthropic SDK's messages.create()
    anthropic_messages = self._prepare_anthropic_messages_for_api(api_messages)
    return build_anthropic_kwargs(model=..., messages=anthropic_messages, ...)

if self.api_mode == "codex_responses":
    # Use OpenAI Responses API (different tools parameter structure)
    payload_messages = api_messages # No system wrapper
    instructions = ""

if self.api_mode == "chat_completions":
    # Standard OpenAI chat completions (messages[0] injects system prompt)
    ...

```

2.3.3 Streaming Response Handling Differences

```

# run_agent.py:3912-3930
if self.api_mode == "codex_responses":
    # Codex stream handled internally by _run_codex_stream
    result["response"] = self._run_codex_stream(api_kwargs, ...)

```

```

if self.api_mode == "anthropic_messages":
    # Anthropic SDK native stream: client.messages.stream()
    ...

```

2.3.4 Credential Refresh Mechanism Differences

API Mode	Credential Refresh Method
anthropic_messages	_try_refresh_anthropic_client_credentials()
chat_completions + Nous	_try_refresh_nous_client_credentials()
codex_responses + OpenAI Codex	_try_refresh_codex_client_credentials()

2.3.5 Mode Switching on Fallback Activation

```

# run_agent.py:4388-4402
fb_api_mode = "chat_completions"
if fb_provider == "openai-codex":
    fb_api_mode = "codex_responses"
elif fb_provider == "anthropic" or fb_base_url.endswith("/anthropic"):
    fb_api_mode = "anthropic_messages"
elif self._is_direct_openai_url(fb_base_url):
    fb_api_mode = "codex_responses"

self.api_mode = fb_api_mode # Synchronized mode switch

```

 Note

Fallback Switch = Joint Constraint Rebinding.

One update changes four properties simultaneously: model, provider, base_url, api_mode. The context_compressor also recalculates threshold_tokens proportionally based on the fallback model's context_length.

Problem: If only the model is updated but context_compressor is forgotten, the primary's 200K threshold will be incorrectly reused for a small-capacity fallback model (32K), causing immediate session overflow.

Key: Related state must be updated together, otherwise the constraint system becomes inconsistent.

 Note

Prompt Caching Enable Conditions = Platform Feature Sniffing.

Actually a three-way joint condition:

- anthropic_messages mode: platform guarantees support, always enabled
- OpenRouter + Claude: requires dual cross-validation (provider name + model name)

Reason: OpenRouter aggregates multiple backends, whether cache primitives are available depends on the **actual routed backend**, not the platform itself. Checking only is_openrouter would incorrectly enable caching for non-Claude models.

3. Memory Mode (Memory Mode)

3.1 Mode Definition

Configuration Location: `honcho_integration/client.py:81–136`, `HonchoClientConfig` class

Memory mode determines how Hermes persists Agent's cross-session memory, **related to the interaction strategy with Honcho cloud memory service**.

Mode	Value	Behavior
Hybrid	hybrid (default)	Write to both Honcho cloud and local <code>MEMORY.md</code> / <code>USER.md</code>
Honcho Only	honcho	Write to Honcho cloud only, disable local markdown file writes
Local Only	local	Use local markdown files only (skip Honcho-related writes)

Note

The three memory modes correspond to three classic consistency models:

- `hybrid` = **Write-Ahead + Write-Through**: write to both copies simultaneously, balancing cloud portability and local readability. Requires try/except compensation if one side fails.
- `honcho` = **Remote Persistence First**: local is read-only cache, CDN origin-first strategy, suitable for multi-device sync.
- `local` = **Local-Write-as-Commit**: zero network dependency, cannot share across devices, single-node database model.

`honcho` mode implements soft-disable rather than hard-deleting code paths — a **Conditional Skip** strategy.

3.2 Configuration Hierarchy and Priority

Honcho configuration supports multi-level overrides, priority from high to low:

```
hosts[hermes].memoryMode  (host-level, explicit config)
    ↓ (if not exists)
global memoryMode          (root-level)
    ↓ (if not exists)
"hybrid"                   (hardcoded default)
```

Also supports **per-peer overrides**:

```
# honcho_integration/client.py:70–78
def _resolve_memory_mode(global_val: str | dict, host_val: str | dict | None) -> dict:
    val = host_val if host_val is not None else global_val
    if isinstance(val, dict):
        default = val.get("default", "hybrid")
        overrides = {k: v for k, v in val.items() if k != "default"}
    else:
        default = str(val) if val else "hybrid"
        overrides = {}
    return {"memory_mode": default, "peer_memory_modes": overrides}

# honcho_integration/client.py:104–109
def peer_memory_mode(self, peer_name: str) -> str:
    return self.peer_memory_modes.get(peer_name, self.memory_mode)
```

Configuration example (JSON format, `honcho.json`):

```
{
  "memoryMode": {
    "default": "hybrid",
    "hermes": "honcho",
    "other-peer": "hybrid"
  }
}
```

3.3 Runtime Behavior Differences

```
# run_agent.py:1119-1131
if self._honcho_config and self._honcho:
    _hcfg = self._honcho_config
    _agent_mode = _hcfg.peer_memory_mode(_hcfg.ai_peer)
    _user_mode = _hcfg.peer_memory_mode(_hcfg.peer_name or "user")

    if _agent_mode == "honcho":
        self._memory_flush_min_turns = 0      # No forced wait, disable directly
        self._memory_enabled = False          # Local MEMORY.md writes completely disabled
        logger.debug("peer %s memory_mode=honcho: local MEMORY.md writes disabled", _hcfg.ai_peer)

    if _user_mode == "honcho":
        self._user_profile_enabled = False     # USER.md writes disabled
        logger.debug("peer %s memory_mode=honcho: local USER.md writes disabled", _hcfg.peer_name
or "user")
```

Note

Two orthogonal write pipelines, asymmetric memory visibility.

MEMORY.md and USER.md are two completely independent state machines: each has its own enabled flag and flush guard, independent of each other.

Effect: Setting peer A (AI side) to `honcho` only disables Agent memory writes; peer B (user side)'s USER.md pipeline is completely unaffected. No "if ai vs user" branching needed in implementation, just relies on two pre-bound independent flags.

Essence: **Separation of Concerns at the data flow level.**

3.4 CLI Management Interface

```
# View current memory mode
hermes honcho mode

# Set memory mode
hermes honcho mode hybrid    # Write to Honcho + local files
hermes honcho mode honcho    # Honcho only (skip local files)

# Interactive setup (wizard)
hermes honcho setup
# Memory mode options:
#   hybrid  - write to both Honcho and local MEMORY.md (default)
#   honcho  - Honcho only, skip MEMORY.md writes
```

4. Memory Recall Mode (Recall Mode)

4.1 Mode Definition

Configuration Location: `honcho_integration/client.py:47–54`, `run_agent.py:2394–2400`

Recall mode controls **how Honcho memory is used at inference time**, i.e., "what memory information can the model see".

Mode	Value	Behavior
Hybrid Recall	hybrid (default)	Auto-inject context + Honcho tools available (model decides)
Context Only	context	Auto-inject context only, Honcho tools removed from tool list
Tools Only	tools	No auto-inject context, but Honcho tools available

 **Note**

Recall mode = the allocation problem of "who bears the retrieval computation".

Computation location:

- `context` → At Agent startup (prefetch) → Latency = Prefetch RTT → Token cost = Fixed
- `tools` → During model inference (on-demand) → Latency = Model decision → Token cost = Variable
- `hybrid` → Both

`context` = **Greedy Prefetch**: assumes memory is always useful, loads upfront. Pros: zero extra latency at inference. Cons: if prefetched content is irrelevant, wastes tokens.

`tools` = **Lazy Loading**: model judges when to use memory on demand. Higher precision, but with call latency.

`hybrid` = Prefetch reduces first-turn latency + preserves flexibility of on-demand tool calls.

4.2 Runtime Behavior

```
# run_agent.py:2394–2400
if hcfg.recall_mode == "context":
    self._strip_honcho_tools_from_surface() # Remove honcho_* tools
    print(" Honcho active – recall_mode: context (Honcho tools hidden)")
else:
    print(f" Honcho active – recall_mode: {hcfg.recall_mode}")

# run_agent.py:6135–6139
_recall_mode = (self._honcho_config.recall_mode if self._honcho_config else "hybrid")
if self._honcho and self._honcho_session_key and _recall_mode != "tools":
    prefetched_context = self._honcho_prefetch(original_user_message)
    if prefetched_context:
        self._honcho_context = prefetched_context # Inject into system prompt
```

4.3 System Prompt Injection Differences

```
# run_agent.py:2613–2631
recall_mode = hcfg.recall_mode if hcfg else "hybrid"
honcho_block = (
    "# Honcho memory integration\n"
    f"Active. Session: {self._honcho_session_key}. "
```

```

    f"Mode: {mode}. Write frequency: {freq}. Recall: {recall_mode}.\n"
)

if recall_mode == "context":
    honcho_block += (
        "Honcho context is injected into this system prompt below. "
        "All memory retrieval comes from this context – no Honcho tools "
        "are available. Answer questions about the user, prior sessions, "
        "and recent work directly from the Honcho Memory section.\n"
    )
elif recall_mode == "tools":
    honcho_block += "Honcho tools:\n  honcho_context <question> – ask Honcho a question, LLM-
synthesized answer\n"
    "  honcho_search – semantic search (no LLM)\n"
    "  honcho_profile – peer card snapshot (no LLM)\n"
    "  honcho_conclude – write conclusion to memory (no LLM)\n"

```

4.4 Honcho Toolset

When `recall_mode != "context"`, the following tools are available to the model:

Tool Name	Function	LLM Call
<code>honcho_context</code>	Ask Honcho a question, get LLM-synthesized answer	Yes
<code>honcho_search</code>	Semantic search over history	No
<code>honcho_profile</code>	Get user peer card snapshot	No
<code>honcho_conclude</code>	Write conclusion/facts to Honcho memory	No

Note

`_strip_honcho_tools_from_surface()` = **Capability Stripping at Initialization.**

Removes `honcho_*` tools from the tool list at initialization, rather than checking permissions at execution time. The model cannot see these tools in the tool list → won't try to call them → no call failures and retry overhead.

More thorough than runtime guards: guards are "saw it but denied", Capability Stripping is "never existed".

Effect: **Tool selection confusion reduced to zero.**

5. Write Frequency Mode (Write Frequency)

5.1 Mode Definition

Configuration Location: `honcho_integration/client.py:110–112`

Write frequency controls **when and how Honcho memory writes occur**.

Mode	Value	Behavior
Async Write	<code>async</code> (default)	Background thread async write, non-blocking, no token cost
Sync Write	<code>turn</code>	Sync write after every turn, blocks until complete
Batch Write at Session End	<code>session</code>	Batch flush only at session end
Write Every N Turns	Integer N	Write once every N turns

Note

Write frequency = batch processing strategy, core trade-off: latency vs throughput vs loss risk.

Mode	Writes/Session	Loss Risk	Suitable Scenario
turn	N	Very Low	Critical info must not be lost
async	~N	Low (retryable)	Daily conversations
session	1	High (crash = loss)	Short sessions/read-only
N (integer)	~session_length/N	Medium	Custom batch size

async mode = **Eventual Consistency + Write-Behind Caching**: writes are not immediately visible to subsequent queries, guaranteed within a bounded time. Background thread is the enqueue end of an SPSC queue, next turn triggers flush → typical async pipeline.

PS:

1. The corresponding rule is determined at session startup — read `write_frequency` from config file, store in `self._write_frequency`
2. Remains unchanged throughout the entire session cycle — `_write_frequency` has no setter, cannot be modified dynamically at runtime
3. Non-adaptive — no dynamic adjustment logic based on system load, network latency, token consumption, etc. in the code
4. Only `async` mode has retry mechanism — background thread retries after 2 seconds on failure, discards after second failure

5.2 Runtime Implementation

```
# run_agent.py:5036-5041
_hcfg = getattr(self, '_honcho_config', None)
if _hcfg and _hcfg.peer_memory_mode(_hcfg.ai_peer) == "honcho":
    return # honcho-only mode skips local flush

effective_min = min_turns if min_turns is not None else self._memory_flush_min_turns
if self._user_turn_count < effective_min:
    return # Insufficient turns, skip flush

# honcho_integration/session.py
# HonchoSessionManager decides based on write_frequency:
#   async → submit to background thread
#   turn  → sync write
#   session → batch write only at shutdown
```

5.3 CLI Management Interface

```
# Interactive setup (hermes honcho setup)
hermes honcho setup
# Write frequency options:
#   async  - background thread, no token cost (recommended)
#   turn   - sync write after every turn
#   session - batch write at session end only
#   N      - write every N turns (e.g. 5)
```

6. Reasoning Level (Dialectic Reasoning Level)

6.1 Mode Definition

Configuration Location: honcho_integration/client.py:116–118

Dialectic Reasoning controls the depth of Honcho AI-to-AI inference, used for context synthesis at session start.

Level	Value	Behavior Description
Minimal	minimal	Least reasoning, almost passthrough
Low	low (default)	Light reasoning, suitable for most scenarios
Medium	medium	Moderate depth reasoning
High	high	High-intensity reasoning
Max	max	Maximum reasoning depth

 **Note**

Dialectic reasoning = "Compute-for-Understanding", an anytime algorithm.

The five levels from `minimal` to `max` determine how many AI-to-AI inferences the Honcho dialectic endpoint performs to synthesize user profiles. Higher reasoning depth = more token consumption, but produces better quality context summaries.

This is a typical **anytime algorithm**: lower levels return approximate results immediately, higher levels continuously improve within time budget. `prefetch_dialectic` is a background thread (non-blocking), results are consumed in the next turn — **async pipeline**.

The truncation logic for `dialectic_max_chars` is noteworthy: `result[:max_chars].rsplit(" ", 1)[0] + " ..."` — finds the nearest space before the character boundary for truncation, ensuring no partial words remain after truncation. **Semantically preserved bounded truncation** rather than hard cut.

6.2 CLI Management Interface

```
# View/set reasoning level
hermes honcho peer                # Display current settings
hermes honcho peer --reasoning medium # Set to medium

# Also set Dialectic cap (max characters injected into system prompt)
hermes honcho tokens --dialectic 800 # Set dialectic result max to 800 characters
```

 **Note**

Dialectic is a cross-session Peer Modeling system.

It's not memory retrieval; rather, Honcho has the AI-side peer and User-side peer conduct an AI-to-AI dialogue to synthesize a high-level summary of "who is the user, what does the user care about, what is the user's work pattern".

In ML terms, this is similar to **dynamic refinement of user embeddings**: at each session end, use conversation history to refine the user's representation, injected into the next session's system prompt. The difference is this is dialogue-driven reasoning, not statistical embedding.

7. Session Strategy

7.1 Mode Definition

Configuration Location: honcho_integration/client.py:127–128

Session strategy determines how the working directory is mapped to Honcho sessions.

Strategy	Value	Behavior
Per-Directory	per-directory (default)	One session per working directory
Per-Session	per-session	Create new session on each run, named by Hermes session ID
Per-Repo	per-repo	One session per Git repository (using repo root directory name)
Global	global	All directories share a single session

 **Note**

per-repo uses the Git repository root directory as the isolation boundary — a **low-cost, high-benefit heuristic strategy** that naturally avoids cross-repository pollution without manual whitelist configuration.

7.2 CLI Management Interface

```
# Session mapping management
hermes honcho sessions          # List known directory→session mappings
hermes honcho map <name>       # Map current directory to specified session name
```

 **Note**

Session name is a deterministic function of working directory, naturally supports session reconstruction.

per-directory → directory name; per-repo → Git repository name; per-session → Hermes session_id. This means **the same physical environment always maps to the same Honcho session**, no explicit mapping table needed.

Manual mapping (hermes honcho map <name>) is a runtime override, higher priority than auto-resolution — allows the same directory to temporarily redirect to a different session context. This is a **Temporary Binding Override**, doesn't change the auto strategy, only changes the current semantics.

8. CLI Display Mode and Interaction Mode

8.1 busy_input_mode

Configuration Location: cli.py:1068–1069 , used by run_agent.py via self.busy_input_mode

Controls "what happens when user presses Enter while Agent is outputting":

Mode	Value	Behavior
Interrupt	interrupt (default)	Enter key sends interrupt signal, interrupts Agent's current run
Queue	queue	Enter key queues input, processed after Agent completes

```
# cli.py:1068–1069
_bim = CLI_CONFIG["display"].get("busy_input_mode", "interrupt")
self.busy_input_mode = "queue" if str(_bim).strip().lower() == "queue" else "interrupt"
```

Note

`busy_input_mode` : Queue mode = bounded buffer, Interrupt mode = drop strategy.

- `queue` mode: Enter key input goes into FIFO queue, processed in order after Agent completes → **bounded queue**, queue full policy is undefined (depends on implementation, may block or drop).
- `interrupt` mode: Enter key directly sends interrupt signal, drops any intermediate state in current run → **interrupt-and-drop**, aggressive but simple.

This is essentially **input handling strategy selection**: queue mode suits long tasks (multiple user inputs not lost), interrupt mode suits short tasks (quick interrupt to restart).

8.2 tool_progress_mode

Configuration Location: `cli.py:1057–1060`

Controls the progress display mode during tool call execution:

Mode	Value	Behavior
off	off / False	Turn off tool progress display
new	new	Show only new tool calls
all	all (default)	Show all tool calls
verbose	verbose	Verbose mode (includes log prefix)

YAML 1.1 parses bare `off` as boolean `False`, code has compatibility handling:

```
_raw_tp = CLI_CONFIG["display"].get("tool_progress", "all")
self.tool_progress_mode = "off" if _raw_tp is False else str(_raw_tp)
```

8.3 resume_display

Configuration Location: `cli.py:1061–1062`

Controls the display mode when resuming historical sessions:

Mode	Value	Behavior
full	full (default)	Show complete history messages
minimal	minimal	Show only single-line summary

8.4 streaming mode

Configuration Location: `cli.py:1073–1074`

Controls whether to stream LLM returned tokens to the terminal:

```
self.streaming_enabled = CLI_CONFIG["display"].get("streaming", False)
```

Note

`streaming_enabled` is essentially token-level streaming delivery vs. batch assembly.

Streaming off → LLM token sequence is fully assembled server-side before being returned in one batch, user sees complete answer; streaming on → each token is pushed to client in real-time (pipeline), user experiences "character-by-character output".

This involves **backpressure** problems: when model generation speed exceeds network bandwidth, streaming pipeline needs to buffer or block. In streaming mode, the latency of each `on_first_delta` callback is the measure of backpressure.

PS: In practice, when the model streams output too slowly or responses are unstable, incomplete content may cause tool/function call parsing failures. In such cases, non-streaming output can be used temporarily to work around this issue.

9. Toolset (Tool Collection) System

9.1 Toolset Definition

Configuration Location: `toolsets.py:72–300`

Toolset is a logical grouping of tools; at Agent initialization, you can specify which toolsets to enable or disable:

```
# Specify toolsets at CLI startup
hermes --toolsets web,terminal
python cli.py --toolsets web,terminal,vision
```

9.2 Built-in Toolsets

Toolset Name	Included Tools	Description
web	web_search , web_extract	Web search and content extraction
search	web_search	Web search only
vision	vision_analyze	Image analysis
image_gen	image_generate	Image generation
terminal	terminal , process	Terminal and process management
file	read_file , write_file , patch , search_files	File operations
browser	browser_navigate , browser_snapshot , ...	Browser automation
memory	todo , memory	Task management and memory
skills	skills_list , skill_view , skill_manage	Skill management
tts	text_to_speech	Text-to-speech
code	execute_code , delegate_task	Code execution and sub-agent delegation
mcp	MCP protocol-related tools	MCP tool ecosystem
honcho	honcho_context , honcho_profile , honcho_search , honcho_conclude	Honcho memory tools
research	Composite toolset	Comprehensive research
development	Composite toolset	Development-related
full_stack	Composite toolset	Full-stack tools

PS: When there are too many tools, you can refer to CC's tool lookup feature for lazy loading.

9.3 Composite Toolsets

Toolsets can include other toolsets:

```

TOOLSETS = {
    "research": {
        "description": "Comprehensive research toolkit",
        "tools": [],
        "includes": ["web", "file", "memory"]
    },
}

```

9.4 Toolset Filtering Logic

```

# run_agent.py:926-947
self.tools = get_tool_definitions(
    enabled_toolsets=enabled_toolsets,    # Only these toolsets
    disabled_toolsets=disabled_toolsets,  # Exclude these toolsets
    quiet_mode=self.quiet_mode,
)

if enabled_toolsets:
    print(f"    ✅ Enabled toolsets: {' '.join(enabled_toolsets)}")
if disabled_toolsets:
    print(f"    ❌ Disabled toolsets: {' '.join(disabled_toolsets)}")

```

Note

Toolset include = DAG composition + topological sort, cycle detection at initialization rather than runtime.

research includes ["web", "file", "memory"] → system parses dependency graph and flattens to final tool list. Expands in dependency order, detects A includes B, B includes A cycles, deduplicates duplicate tools.

Detecting conflicts at initialization → **impossible** for agent to discover tool unavailable mid-execution.

Note

enabled + disabled = whitelist first, blacklist fallback: `whitelist n ~blacklist`.

When whitelist exists, only toolsets in the whitelist are enabled; blacklist excludes specified items on top of that. Same mechanism as Linux package manager's `apt install X + apt exclude Y`.

Benefit: fine-grained adjustment on top of predefined toolsets, without redefining the entire set.

10. Plan Mode (/plan Command)

10.1 Architecture Overview

Important clarification: `/plan` is not strictly an independent Agent run mode (like "Plan Mode: On"), but rather a **slash command** that executes a specialized planning workflow by loading the `/plan` skill.

Core understanding:

- `/plan` is essentially a **skill loading mechanism**, not an independent Agent mode
- When triggered, it packages the contents of the `plan` skill into a system message, injected into the Agent's reasoning pipeline

- Skill content + `runtime_note` jointly guide the model to execute the planning workflow and output to the specified file

10.2 Current Trigger Method (Manual)

CLI Handling (`cli.py:4030–4054`):

```
def _handle_plan_command(self, cmd: str):
    """Handle /plan [request] – load the bundled plan skill."""
    parts = cmd.strip().split(maxsplit=1)
    user_instruction = parts[1].strip() if len(parts) > 1 else ""

    plan_path = build_plan_path(user_instruction)    # Generate default plan file path
    msg = build_skill_invocation_message(
        "/plan",
        user_instruction,
        task_id=self.session_id,
        runtime_note=(
            "Save the markdown plan with write_file to this exact relative path "
            f"inside the active workspace/backend cwd: {plan_path}"
        ),
    )
    # Put message in input queue, wait for Agent reasoning to process
    self._pending_input.put(msg)
```

Gateway Handling (`gateway/run.py:1900–1920`):

```
if canonical == "plan":
    from agent.skill_commands import build_plan_path, build_skill_invocation_message

    user_instruction = event.get_command_args().strip()
    plan_path = build_plan_path(user_instruction)    # Generate default plan file path
    event.text = build_skill_invocation_message(
        "/plan",
        user_instruction,
        task_id=_quick_key,
        runtime_note=(
            "Save the markdown plan with write_file to this exact relative path "
            f"inside the active workspace/backend cwd: {plan_path}"
        ),
    )
    canonical = None # Clear, skip normal command handling
```

Common pattern on both ends:

1. Parse user instruction
2. Call `build_plan_path()` to generate plan file path
3. Call `build_skill_invocation_message()` to load skill and construct activation message
4. Inject message into Agent's reasoning pipeline

10.3 Plan File Path Generation Rules

```
# agent/skill_commands.py:21–39
def build_plan_path(user_instruction: str = "", *, now: datetime | None = None) -> Path:
```

```
slug_source = (user_instruction or "").strip().splitlines()[0] if user_instruction else ""
slug = _PLAN_SLUG_RE.sub("-", slug_source.lower()).strip("-")
if slug:
    slug = "-".join(part for part in slug.split("-")[:8] if part)[:48].strip("-")
slug = slug or "conversation-plan"
timestamp = (now or datetime.now()).strftime("%Y-%m-%d_%H%M%S")
return Path(".hermes") / "plans" / f"{timestamp}-{slug}.md"
```

Generated format: `.hermes/plans/<year-month-day_hourminsec>-<slug>.md`

10.4 Skill Loading Mechanism

```
# agent/skill_commands.py:220-255
def build_skill_invocation_message(cmd_key: str, ...):
    commands = get_skill_commands() # Scan ~/.hermes/skills/
    skill_info = commands.get(cmd_key) # Find /plan skill
    loaded = _load_skill_payload(skill_info["skill_dir"], task_id=task_id)
    # Construct activation message: includes [SYSTEM: user invoked "plan" skill] + skill content
```

10.5 Possibility of Auto-Triggering

While `/plan` is currently a manually-triggered slash command, the underlying mechanism makes it capable of auto-triggering:

Method 1: Hook Mechanism Auto-Trigger

The project supports hooks configuration (`.claude/settings.json`), enabling automation rules:

```
{
  "hooks": {
    "on_user_message": [
      "when: pattern matches 'implement|build|create|添加功能|实现'",
      "then: auto_invoke /plan"
    ]
  }
}
```

Method 2: Rule-Based Auto-Trigger

Add detection logic in `cli.py`, auto-invoke when user request matches specific patterns:

```
# cli.py command handling logic auto-trigger point
def _auto_detect_plan_trigger(self, user_input: str) -> bool:
    """Detect whether plan mode should be auto-triggered"""
    trigger_patterns = [
        r"implement",
        r"build.*feature",
        r"添加.*功能",
        r"实现.*需求",
        # More patterns...
    ]
    return any(re.search(p, user_input.lower()) for p in trigger_patterns)
```

Method 3: Modify System Prompt for Model Self-Decision

Add instructions in system prompt, letting the model proactively suggest `/plan` when detecting scenarios requiring planning:

```
# config.yaml
system_prompt_overrides:
  - when: "task complexity > threshold"
    inject: "Hint: This task may benefit from planning. Consider suggesting /plan."
```

Note

`/plan` trigger mechanism = Lazy Command Loading.

The essential difference between `/plan` and other slash commands: it doesn't execute a specific action (like `/help` printing help), but **loads a complete skill workflow** injected into the reasoning pipeline.

This means `/plan`'s "execution" is actually completed by the model itself — skill content guides the model to think according to the planning process, `runtime_note` tells the model where to output. The command handler only prepares input data.

From this perspective, **automating `/plan` trigger only needs to change the timing of "when to prepare input data"**, not the execution logic itself.

11. Sub-Agent Delegation Mode (Delegation)

11.1 Delegation Parameters

Configuration Location: `run_agent.py:5319–5328`, `tools/delegate_tool.py`

Create sub-agents via `delegate_task` tool, supports the following parameters:

Parameter	Type	Description
<code>goal</code>	<code>str</code>	Sub-agent goal description
<code>context</code>	<code>str</code>	Additional context
<code>toolsets</code>	<code>list[str]</code>	Toolsets available to sub-agent (default <code>[terminal, file, web]</code>)
<code>tasks</code>	<code>list</code>	Sub-task list
<code>max_iterations</code>	<code>int</code>	Sub-agent max tool call turns (default 45)

11.2 Delegation Configuration

```
# config.yaml
delegation:
  max_iterations: 45          # Sub-agent max turns
  default_toolsets:          # Sub-agent default toolsets
    - terminal
    - file
    - web
  model: ""                  # Sub-agent model (empty = inherit parent model)
  provider: ""               # Sub-agent provider (empty = inherit)
  base_url: ""               # Direct OpenAI compatible endpoint
  api_key: ""                # Endpoint API key
```

Note

Sub-agent = Fork-Join parallel model, `max_iterations` = depth limit to prevent infinite recursion.

After `delegate_task`: parent process (waiting for sub-agent result) and child process (independent execution) fork → child completes → parent collects result and continues. Standard Fork-Join.

`max_iterations: 45` = **depth limit for depth-first search**, prevents infinite recursion in complex tasks.

Two Agents share parent process's context injection (`context` parameter), but each has independent toolsets and model configuration → **restricted shared memory model**: avoids global state contention while allowing subtasks to have independent capability boundaries.

Note

Sub-agent default toolsets `[terminal, file, web]` = Principle of Least Privilege.

Parent Agent defaults to all toolsets, sub-agent can only operate on terminal, files, network. Cannot call `delegate_task` (prevents nested delegation) or `honcho_*` (prevents memory cross-contamination).

More capabilities → must explicitly pass `toolsets` parameter → forces developers to **consciously choose** sub-agent capability boundaries, rather than defaulting to fully open.

12. Platform Mode (Platform Modes)

12.1 Supported Platforms

Configuration Location: `run_agent.py:546–547`, `AIAgent.platform` attribute

Platform	Value	Description
CLI	<code>cli</code> (default)	Command line interface
Telegram	<code>telegram</code>	Telegram messaging platform
Discord	<code>discord</code>	Discord platform
WhatsApp	<code>whatsapp</code>	WhatsApp platform
Feishu	<code>feishu</code>	Feishu platform
Mattermost	<code>mattermost</code>	Mattermost platform
Others	Various strings	Extended platforms

12.2 Platform-Aware Behavior

Platform information is injected into system prompt, affecting formatting hints:

```
# run_agent.py:546–547
self.platform = platform # "cli", "telegram", "discord", "whatsapp", etc.
# Used to inject platform-specific formatting hints
```

Note

Platform is just a string identifier, adapts behavior via system prompt rather than code paths.

Adding a new platform (without modifying Agent core logic): just inject a formatted system prompt segment telling the model "You're on Telegram, messages have length limits, responses should not be too long", etc. Platform identifier is an **open set** — code doesn't enumerate all platforms, accepts any string.

This is **data-driven behavior adaptation**: behavioral differences come from prompt content, not code branches. If platform differences require deeper changes beyond prompts (such as mandatory validation of message length limits), only then would actual code branching be needed — currently no such requirement exists in the code.

13. Other Configuration Dimensions

13.1 Reasoning Configuration

```
# run_agent.py:541-542
reasoning_config: Dict[str, Any] = None # Override reasoning config
# Example: {"enabled": True, "effort": "medium"}
# OpenRouter: effort=none disables thinking
# Default: {"enabled": True, "effort": "medium"}
```

13.2 Compression Mode

```
# config.yaml
compression:
  enabled: True           # Enable context compression
  threshold: 0.50         # Trigger at 50% context limit
  summary_model: ""       # Summary model (empty = use main model)
```

Note

threshold: 0.50 = watermark trigger, not passive response.

Start preparing compression when context reaches 50%, not waiting until 90%+. Reason: compression operation itself involves summary model call, has extra latency — must trigger early to complete before context is exhausted, imperceptible to user.

Essence: **Proactive water level management**, preventing bottleneck operations from dragging down the entire pipeline.

Additional design: `summary_model` can be independently configured → compression flow is a **dual-model pipeline**: main model handles current task, summary model asynchronously processes historical context.

13.3 Smart Model Routing

```
# config.yaml
smart_model_routing:
  enabled: False
  max_simple_chars: 160    # Simple query character limit
  max_simple_words: 28     # Simple query word limit
  cheap_model: {}         # Cheap model configuration
```

Note

Smart model routing = cost-aware heuristic scheduler.

`max_simple_chars: 160` and `max_simple_words: 28` are heuristic definitions of simple queries: both dimensions below threshold → route to cheaper model.

Weakness: length doesn't equal complexity, this way of judging task difficulty is quite rough.

13.4 Approval Mode

```
# config.yaml
approval:
  auto_approved: []          # Auto-approved tool whitelist
  auto_denied: []           # Auto-denied tool blacklist
  max_tool_calls: 50        # Max RPC calls per turn
```

Note

`max_tool_calls: 50` = safety valve to prevent infinite loops, corresponds to longest path limit of the graph.

Execution graph: user message → Agent reasoning → tool call → tool result → Agent reasoning → ... → may extend infinitely. `max_tool_calls` hard truncates graph depth → forces final answer within 50 steps.

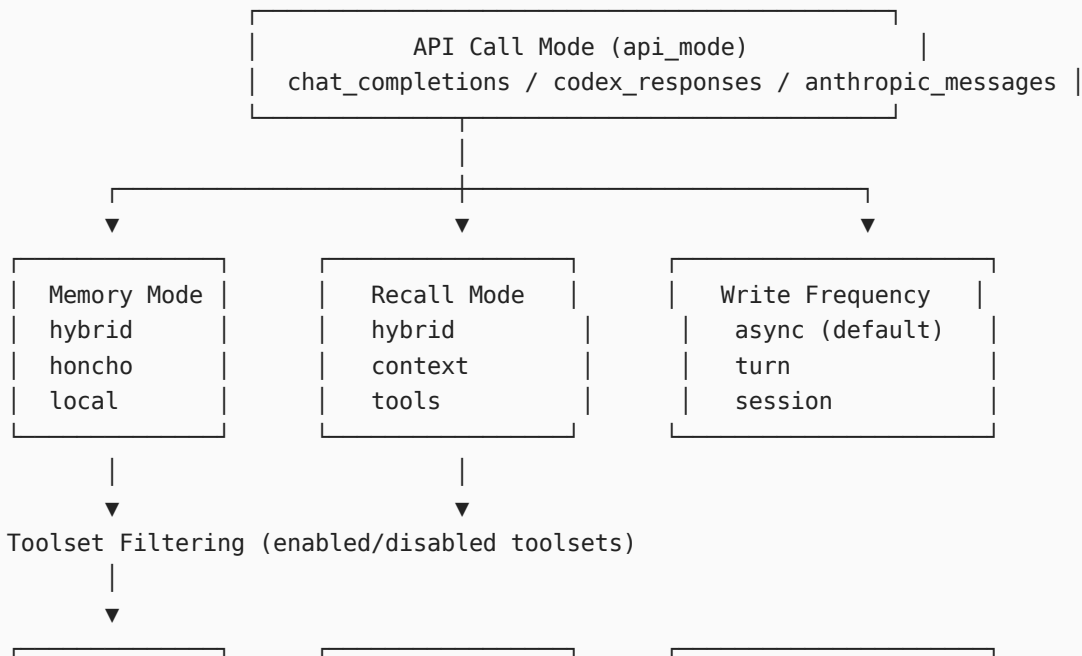
Analogy: **Iterative Deepening** — search within depth limit, report failure rather than continue deeper if solution not found.

Combined with `auto_approved` / `auto_denied` whitelist/blacklist, forms a **hierarchical access control matrix**: high-risk tools (file write, process execution) require human approval; low-risk tools (search, read-only) auto-allow.

13.5 Honcho Doctor Tool Mode

```
# Diagnostic output includes all run mode configurations
hermes doctor
# Output example:
# workspace=xxx mode=hybrid freq=async
```

14. Mode Interaction Matrix



busy_input	tool_progress	Platform Mode
interrupt	off/new/all	cli/telegram/discord
queue	verbose	whatsapp/feishu...

15. Key Code Index

Mode Dimension	Core Definition Location	Runtime Activation Location
API Call Mode	run_agent.py:587–609	run_agent.py:4617 (<code>_build_api_kwargs</code>)
Memory Mode	honcho_integration/client.py:99	run_agent.py:1119–1131
Recall Mode	honcho_integration/client.py:125	run_agent.py:2394, 6135
Write Frequency	honcho_integration/client.py:112	honcho_integration/session.py
Reasoning Level	honcho_integration/client.py:118	honcho_integration/session.py
Session Strategy	honcho_integration/client.py:127	honcho_integration/client.py:resolve_session_na
busy_input_mode	cli.py:1068–1069	cli.py REPL event loop
tool_progress_mode	cli.py:1057–1060	cli.py display logic
Plan Mode	gateway/run.py:1900–1920	agent/skill_commands.py
Toolset Filtering	run_agent.py:654–655	run_agent.py:926–947

16. Summary

Hermes Agent's "run modes" are not a single concept, but a **multi-dimensional orthogonal configuration space**:

- API Layer:** Choose dialogue protocol via `api_mode` (OpenAI Chat, OpenAI Responses, Anthropic Messages)
- Memory Layer:** `memory_mode` × `recall_mode` × `write_frequency` — three dimensions controlling Honcho cloud memory read/write strategy
- Tool Layer:** Toolset filtering determines which tools the Agent can call
- Interaction Layer:** `busy_input_mode`, `tool_progress_mode`, `streaming`, etc. control human-computer interaction experience
- Workflow Layer:** `/plan` and other slash commands trigger specialized skill workflows
- Platform Layer:** Platform identifier affects system prompt formatting and message handling

These dimensions can be freely combined, enabling Hermes to handle scenarios ranging from CLI interaction (`chat_completions` + `hybrid` `memory` + `async` `write`) to Telegram Bot (`chat_completions` + `context` `recall` + Telegram platform) to highly isolated development environments (`anthropic_messages` + `honcho` `memory` + `local` `session` `strategy`).

Note

Overall, Hermes Agent's mode design embodies three core architectural philosophies:

1. Separation of Concerns:

Each dimension controls one orthogonal concern — API protocol, memory storage, memory retrieval, tool capabilities, human-computer interaction, platform adaptation — each evolving independently. Adding a new platform (like a new IM tool) only requires injecting a platform identifier, no core logic changes.

2. Lazy Binding:

Many decisions are deferred to runtime determination — API mode auto-detected via URL fingerprint, recall mode

implements soft-delete via conditional skip, Toolset dynamically flattens via include DAG. Reduces hardcoded dependencies at startup, improves configurability and extensibility.

3. Layered Security Model:

`approval.auto_denied` blacklist, Toolset capability filtering, `max_tool_calls` depth limit, sub-agent least-privilege default toolsets — security control distributed across multiple layers. Even if one layer is bypassed, other layers still provide protection.

Algorithmic perspective: adaptive workflow engine. The system dynamically senses the environment across dimensions (URL fingerprint, model name, platform type), schedules under multiple constraints (token budget, latency, security), and seeks balance among multiple objectives (low latency, low cost, high reliability).